



VIOLATION MANAGEMENT SYSTEM

Project Title	<i>F-Code TrainC Violation Management System</i>
Programming Language	C (C17 standard)
Organizer	Academic Board - F-Code Club, FPT University HCM

Project Period	17 April 2026 - 17 May 2026
Mentor	<i>Nguyen Khoa</i>
Team Name	<i>mid</i>
Submission Date	29 May 2026

Team members

No.	Full Name	Student ID	Role
1	Nguyen Ngoc Phuc	SE203055	Member
2	Dam Le Tuan Anh	SE204111	Team Leader
3	Nguyen Van Phu	SE210043	Member
4	Huynh Gia Bao	SE204913	Member
5	Vo Hieu Thang	SE200932	Member

Approved by **Pham Hoang Tuan**
Academic Board Representative - F-Code Club, FPT University HCM

Table of Contents

Section 1. Project Overview	4
1.1. Problem Statement	3
1.2. Objectives	3
1.3. Scope	4
Section 2. Progress Summary	5
Section 3. System Design	7
3.1. Directory & File Structure	7
3.2. Data Structures	8
3.2.1. Struct Vehicle — stores information about one vehicle	8
3.2.2. Struct PriceConfig — stores fee rates per vehicle type	8
3.2.3. Struct Violation — single violation record	9
3.2.4. Struct AppDatabase — global system state	10
3.3. Key Logic Implemented So Far	10
3.3.1. Vehicle entry validation	10
3.3.2. Fee calculation	10
3.3.3. File persistence strategy	10
3.3.4. Out CLB threshold tracking	11
3.3.5. Member CRUD with cascade operations	11
Section 4. Challenges & Solutions	12
Section 5. Remaining Work	13
Section 6. GitHub Repository	13

Section 1. Project Overview

1.1. Problem Statement

F-Code is one of the most active student clubs at FPT University HCM, with over 50 members distributed across four teams: Academic, Planning, Human Resources, and Media. Managing member discipline has proven difficult without a dedicated system:

- There is no automated way to record violations (missing meetings, not wearing the club jacket, violence, skipping activities). Manual tracking via spreadsheets or paper is error-prone and hard to reconcile.
- Fine amounts vary by role — regular members pay 20,000 VND while team leaders and BCN members pay 50,000 VND — but without automated calculation, totals are often incorrect.
- The Out CLB policy (3 consecutive absences = warning, >3 = removal from club) requires precise tracking that is currently done manually.
- Member, violation, and account data live in separate places, making it difficult to generate reports or aggregate statistics.

The club's Ban Chu Nhiem (BCN) committee needs a terminal-based program that can automatically record violations, calculate fines, warn when members approach the Out CLB threshold, and export reports.

Constraints:

- Maximum 1,000 members and 10,000 violation records at any given time.
- Each member is uniquely identified by their student ID (MSSV). Duplicates are not allowed.
- Data must be persisted to binary files (.dat) after every write operation to prevent data loss on unexpected termination.
- No graphical interface (GUI), no external libraries beyond the C standard library (stdio.h, stdlib.h, string.h, time.h).

1.2. Objectives

Upon completing this project, the team has practiced and improved the following skills:

- Using structs to model real-world entities (Member, Violation, Account) with multiple attributes.
- Managing fixed-size arrays to store and manipulate collections of records.
- Implementing binary file I/O (fopen, fread, fwrite, fclose) so that data persists across program restarts.
- Splitting a program across multiple .h / .c files (auth, member, violation, fileio, utils) for maintainability and modularity.
- Collaborating as a team using Git — committing frequently with meaningful messages, following a story-driven development workflow.

1.3. Scope

In scope (must be delivered by Final Report):

- Login / logout system with account lockout after 3 consecutive failed attempts.
- Self-service password change and BCN-initiated password reset.
- Full member CRUD: add, edit, delete, view profile and member list.
- Violation recording with 4 reasons (No Jacket, Absent, No Activity, Violence).
- Automatic fine calculation based on member role (20,000 / 50,000 VND) and Out CLB handling.
- Out CLB threshold warnings (3 consecutive absences = warning, ≥ 4 = removal prompt).
- Binary file persistence (.dat); data is reloaded on program startup.
- Mark violations as paid and view personal violation history.
- Statistics by team and sort members by violation count.
- Export report to .txt file and search violations by date range.

Out of scope (explicitly excluded):

- Graphical user interface (GUI).
- Network connectivity or concurrent multi-user access.
- External databases or libraries outside the C standard library.

Section 2. Progress Summary

The table below shows the implementation status of all planned features as of the Midterm submission (29/04/2026). Replace the example rows with your team's actual feature list and statuses.

No.	Feature	Priority	Status	Notes
1	Project scaffold & build system	Required	Done	Makefile, clang-format, clang-tidy
2	Data types & constants (types.h)	Required	Done	Account, Member, Violation, AppDatabase
3	Utility functions (utils.c)	Required	Done	Input, validation, time, display helpers
4	File I/O: load/save binary files	Required	Done	Atomic write with backup recovery
5	Authentication: login/logout/session	Required	Done	Lock after 3 fails, session management
6	Change & Reset password	Required	Done	Self-service + BCN reset to MSSV
7	Add member with auto account creation	Required	Done	CRUD + validation + auto-create account
8	Edit member info & recalculate fines	Required	Done	Sync Account.role on role change
9	Delete member & cascade cleanup	Required	Done	Remove account + violations + member
10	View profile & member list	Required	Done	Paginated table with formatting
11	Record violation with fine calculation	Required	Done	Role-based fines, violence handling
12	Out CLB warning & enforcement	Required	Done	3-warning / 4-confirmation threshold
13	Mark paid & view own violations	Required	Not Started	Planned for Story 3.3

14	View all violations with filters	Required	Not Started	Planned for Story 3.4
15	Statistics by team	Required	Not Started	Planned for Story 4.1
16	Sort members by violation count	Advanced	Not Started	Planned for Story 4.2
17	Export report to .txt	Advanced	Not Started	Planned for Story 4.3
18	Search violations by date range	Optional	Not Started	Planned for Story 4.4

Status legend:

- **Done** — feature is fully implemented and manually tested with multiple cases.
- **In Progress** — partial implementation exists; core logic works but some cases are incomplete.
- **Not Started** — planned but no code written yet.
- **Summary:** 12/18 features completed (67%). All core features (Epic 1–3: Authentication, Member CRUD, Violation Recording, Out CLB) are done. The remaining features belong to Epic 3.3–4.4 (statistics, reporting, search) and are expected to be completed before 17/05/2026.

Section 3. System Design

3.1. Directory & File Structure

The project is organized into independent modules, each with a public API header (.h) and an implementation file (.c):

File / Directory	Role / Responsibility
src/main.c	Entry point. Displays the main menu (member or BCN) and routes to the appropriate module.
src/auth.c + include/auth.h	Login, logout, session management, self-service password change, and BCN password reset.
src/member.c + include/member.h	Add, edit, delete members; view personal profile and full member list.
src/violation.c + include/violation.h	Record violations, calculate fines by role, warn and enforce Out CLB.
src/fileio.c + include/fileio.h	Read/write binary data (.dat) for 3 files: members.dat, violations.dat, accounts.dat. Uses atomic write with backup recovery.
src/utlis.c + include/utlis.h	Shared helpers: input reading (read_string, read_int), validation (email, MSSV), time formatting, team/role name mappers.
include/types.h	Defines all structs (Account, Member, Violation, AppDatabase) and domain constants. Single source of truth for data shapes.
Makefile	Build system: compiles with gcc under strict C17 flags (-Wall -Wextra -Wpedantic ...), includes clang-format and clang-tidy targets.
data/*.dat	Binary files storing persisted data: members.dat, violations.dat, accounts.dat.
docs/stories/	Individual story files (1.1 through 4.4) with acceptance criteria, tasks, and post-implementation fix notes.

If your structure is different, update this table to reflect your actual layout. There is no single correct way — consistency and clarity matter more than matching this example exactly.

3.2. Data Structures

The system uses 4 main structs, all defined in include/types.h:

3.2.1. Struct Vehicle — stores information about one vehicle

Purpose: Stores authentication information for each user. Every member has a corresponding Account, created automatically when a new member is added.

Field Name	Data Type	Description
studentId	char[10]	Student ID used as the primary key for login. Example: "SE211001".
password	char[20]	Login password. Defaults to the student's MSSV when the account is first created.
role	int	Access level: 0 = Member, 1 = BCN (Ban Chu Nhiem). BCN role unlocks the admin menu.
isLocked	int	Lock status: 0 = Normal, 1 = Locked (after 3 consecutive failed login attempts).
failCount	int	Number of consecutive failed login attempts. Reset to 0 on successful login.

3.2.2. Struct PriceConfig — stores fee rates per vehicle type

Purpose: Stores personal information and status of a club member. A Member record is created when BCN adds a new member.

Field Name	Data Type	Description
studentId	char[10]	MSSV, primary key linking to Account and Violation records.
fullName	char[50]	Full name of the member.
email	char[50]	Email address. Validated to contain '@' followed by '.'.
phone	char[15]	Phone number.
team	int	Club team: 0 = Academic, 1 = Planning, 2 = HR, 3 = Media.
role	int	Position: 0 = Member, 1 = Leader/Vice, 2 = BCN. Determines fine amount (20k vs 50k).
isActive	int	Status: 1 = Active, 0 = Out CLB (removed from club).

consecutiveAbsences	int	Number of consecutive absences. Reset to 0 when the member attends or commits a non-absence violation. Used for Out CLB threshold checks.
violationCount	int	Total number of violations recorded for this member.
totalFine	double	Total unpaid fine amount (VND). Recalculated when role changes.

3.2.3. Struct Violation — single violation record

Purpose: Records a single violation incident for a member, including timestamp, reason, fine amount, and payment status.

Field Name	Data Type	Description
studentId	char[10]	MSSV of the member who committed the violation.
violationTime	time_t	Timestamp when the violation was recorded, captured via time(NULL).
reason	int	Violation reason: 0 = No Jacket, 1 = Absent, 2 = No Activity, 3 = Violence.
fine	double	Fine amount (VND). Member = 20,000; Leader/BCN = 50,000. Violence: fine = 0.
penalty	int	Penalty type: 0 = Fine (PENALTY_FINE), 1 = Out CLB (PENALTY_OUT_CLB).
isPaid	int	Payment status: 0 = Unpaid, 1 = Paid.
note	char[100]	Optional note added by BCN.

3.2.4. Struct AppDatabase — global system state

Purpose: The single container holding all in-memory system data. It is passed as a parameter to all handler functions instead of using global variables, making the data flow explicit and testable.

Field Name	Data Type	Description
members	Member[1000]	Array storing member records. Max 1,000.
violations	Violation[10000]	Array storing violation records. Max 10,000.

accounts	Account[1000]	Array storing login accounts. 1:1 mapping with Members.
memberCount	int	Current number of members in the array.
violationCount	int	Current number of violations in the array.
accountCount	int	Current number of accounts in the array.

3.3. Key Logic Implemented So Far

3.3.1. Vehicle entry validation

The system uses a 3-step atomic write strategy to ensure data integrity even on unexpected termination (crash):

- Write data to a temporary file (.dat.tmp).
- Rename the current data file to a backup (.dat.bak).
- Rename the temporary file to the data file (.dat). Delete the backup.

On startup, the system checks for leftover .tmp files. If a .tmp exists but the .dat file is missing, the system recovers from the .tmp. If both exist, the .dat is considered valid and the .tmp is discarded.

3.3.2. Fee calculation

The system maintains two independent role hierarchies:

- Account.role (0 = Member, 1 = BCN): Determines menu access. BCN users see the full admin menu; regular members see a limited personal menu.
- Member.role (0 = Member, 1 = Leader, 2 = BCN): Determines fine amounts (20,000 VND for Members, 50,000 VND for Leaders/BCN).

Session state is managed through a static variable inside auth.c, never exposed to other modules. External code accesses the session only through the authGetSession() function.

3.3.3. File persistence strategy

When recording a violation, the system handles each reason type differently:

- REASON_ABSENT: Increments consecutiveAbsences. Runs the Out CLB threshold check immediately after recording.
- REASON_VIOLENCE: Sets fine=0, penalty=PENALTY_OUT_CLB. Prompts BCN for explicit confirmation before marking the member as Out CLB.
- REASON_NO_JACKET / REASON_NO_ACTIVITY: Calculates fine based on role, resets consecutiveAbsences to 0 (member was present).

All data changes (violations, members, accounts) are persisted to files immediately. If a file save fails, the system rolls back in-memory changes to maintain data consistency.

3.3.4. Out CLB threshold tracking

The system automatically tracks each member's consecutive absence streak via the consecutiveAbsences field:

- 3 consecutive absences: Displays a warning that one more absence will result in Out CLB.
- ≥ 4 consecutive absences: Announces the member has exceeded the threshold and prompts BCN for confirmation. If confirmed, isActive is set to STATUS_OUT_CLB.
- Member attends or commits a non-absence violation: Resets consecutiveAbsences to 0.

BCN can also check all members approaching the Out CLB threshold via the "Check Out CLB Threshold" menu, which displays a status table (Monitoring / WARNING / EXCEEDED / Out CLB).

3.3.3. Member CRUD with cascade operations

All member operations ensure data consistency across the three data arrays:

- Add member: Automatically creates a corresponding Account with default password = MSSV.
- Edit member: If the role changes, automatically syncs Account.role and recalculates fines for all unpaid violations.
- Delete member: Cascade-removes the Account, all related Violations, and the Member record. Prevents BCN from deleting their own account.

Section 4. Challenges & Solutions

During development, the team encountered several technical difficulties. The table below summarizes each issue and how it was resolved:

Challenge	Solution / Approach
Data loss on unexpected termination (crash)	Implemented a 3-step atomic write: write .tmp → rename .dat to .bak → rename .tmp to .dat. Added recovery from .tmp on startup. This ensures a crash never corrupts data.
strncpy does not guarantee null-termination	strncpy only appends '\0' if the source string is shorter than the buffer. Fixed by adding buf[MAX_LEN-1] = '\0' after every strncpy call. This is a common C pitfall that is very hard to debug.
Inconsistent saves when multiple files need updating	Defined a specific save order per operation. For example, delete member: accounts → violations → members. If violations save succeeds but members save fails, member stats are rolled back in memory.
Two separate role systems (Account.role vs Member.role)	Account.role only controls menu access (Member/BCN). Member.role determines fine amounts (20k/50k) and has 3 values (Member/Leader/BCN). When editing a member's role, Account.role is automatically synced.
Out CLB threshold: duplicate file saves mid-flow	Separated concerns: violationCheckOutThreshold only updates memory (no file I/O). The caller (violationRecord) is responsible for persistence. This prevents double-saving and potential inconsistency if the second save fails.
Table formatting breaks with long strings	Used precision specifiers in printf: "%-10.10s" to truncate column output. This completely eliminates the case where a string exceeds its column width and misaligns the table.
Login exits immediately on wrong password	authLogin initially returned -1 on wrong password, not allowing retry. Fixed by adding a while(1) loop that retries until either login succeeds or the account is locked (3 failed attempts).

Section 5. Remaining Work

The following features still need to be completed before the Final Report deadline (17/05/2026):

- **Mark paid & view own violations (Story 3.3)** — BCN marks a violation as paid. Members view their personal violation history and outstanding fine total. File: violation.c
- **View all violations with filters (Story 3.4)** — BCN views the full violation list with filters (by MSSV, by reason, by payment status). File: violation.c
- **Statistics by team (Story 4.1)** — Aggregate total fines per team (Academic, Planning, HR, Media). File: violation.c or new report.c
- **Sort members by violation count (Story 4.2)** — Sort the member list by violation count in descending order. File: member.c or new report.c
- **Export report to .txt (Story 4.3)** — Generate a structured text report of violations and fines. File: new report.c
- **Search violations by date range (Story 4.4)** — Search for violations within a date range (from date — to date). File: violation.c or new report.c
- **End-to-end testing** — Run a full scenario: add member → login → record violation → Out CLB warning → edit/delete → export report.

Estimated completion confidence: High — All core features are complete and tested. The remaining features (3.3–4.4) are independent additions that do not depend heavily on existing logic. The violation.c module already has helper functions (fine calculation, MSSV filtering) that will speed up extension. The biggest risk is Story 4.3 (export report) since it requires creating a new report.c module from scratch.

Section 6. GitHub Repository

Repository URL	https://github.com/dltuananh123/fcode-trainc-violation-management-system
Main branch	main
README	Yes — Detailed setup instructions (MSYS2, gcc, clang-format, clang-tidy), build and run commands.
Compile command	mingw32-make (Windows cmd, uses Makefile with gcc)
Compiler flags	-std=c17 -m64 -Wall -Wextra -Wpedantic -Wshadow -Wconversion -Wformat=2 -Wformat-security -Wfloat-equal -Wundef -Wsign-conversion -Wcast-align -Wcast-qual -Wmissing-prototypes -Wmissing-declarations -Wunreachable-code -Wnull-dereference -Wimplicit-fallthrough -Wswitch-enum -Wpointer-arith -linclude
Development branch	feat-story-1.1-project-scaffold (current)

Note: The repository is set to Public. All commit history and story documentation are accessible. The team follows a story-driven development workflow: each story has its own markdown file in docs/stories/ with acceptance criteria, tasks, and post-implementation fix notes.